

ECS DOTS: когда стандартный Unity сдаётся

Серия «ECS DOTS на практике», статья 1 из 4 — концептуальное введение

Часть 1: Зачем вообще ECS DOTS, если есть привычный MonoBehaviour?

Я всё чаще прихожу к ответу: писать игровую логику нужно не только так, как мы привыкли последние 15 лет в Unity. MonoBehaviour-подход прекрасно работает для UI, меню и одиночных объектов — но как только на сцене появляются сотни однотипных сущностей (враги, projectiles, particles), он начинает заваливаться: GC-спайки, дёрганный фрейм-тайм, перегретое устройство.

А вот понять, **где провести границу архитектуры**: что оставить на MonoBehaviour, что переписать на ECS, какие данные класть рядом в памяти, а какие — нет, — это всё ещё инженерное мышление, которое AI за тебя не сделает.

И особенно хорошо это видно в **performance-critical Unity-проектах**.

Поэтому хочу сделать небольшую серию постов про темы, которые, на мой взгляд, стоит понимать каждому Unity-разработчику, который хоть раз слышал слова «60 FPS на мобилке» или «1000 юнитов одновременно». Не в формате «выучил термин — ответил», а в формате «понимаешь ли ты, какие последствия у твоего архитектурного выбора».

Поехали.

Контекст: откуда вообще тема

Делал pet-проект — top-down survivors клон с целью **500+ врагов одновременно на экране** при таргете 60 FPS на мобильном устройстве. Стандартный MonoBehaviour-подход начал заваливаться уже на 80–100 врагах.

Решением стал переход на **ECS DOTS** — официальный Data-Oriented стек Unity. Эта серия из 4 статей разбирает, что я понял в процессе и как это применять в production.

В первой статье — **фундаментальное «почему»**. Без кода, без туториалов. В чём принципиальная разница ECS DOTS и классического MonoBehaviour, и в каких ситуациях эта разница того стоит.

Разница ECS DOTS и классической архитектуры Unity

Это главное, что нужно понять, прежде чем лезть в код. Дальше всё — следствия.

Классика: MonoBehaviour = объекты с поведением

Привычный Unity — это **Object-Oriented**. Каждый игровой объект — это GameObject + набор MonoBehaviour-компонентов. Данные и поведение **лежат в одном классе**:

```
class Enemy : MonoBehaviour {
    public float health;      // данные
    public float speed;       // данные
    void Update() { ... }     // поведение
}
```

Каждый **Enemy** — это managed-объект на куче. У него своя ссылка на Transform, своя vtable, свой набор служебных полей. Unity каждый кадр обходит **список всех активных скриптов** и через виртуальный вызов дёргает **Update()** у каждого.

Это работает, пока объектов десятки. На сотнях — начинаются проблемы.

ECS DOTS: данные отдельно, логика отдельно

ECS разделяет три понятия, которые в Mono слиплись в одном классе:

- **Entity** — это просто целое число (ID). Никаких полей, никакого поведения.
- **Component** — структура с данными. Просто **struct**, без методов.
- **System** — функция, которая каждый кадр обрабатывает entities с заданным набором компонентов.

Игра в ECS — это **трансформации над массивами данных**, а не сообщения между объектами. Никаких классов, наследования, виртуальных методов.

Главное архитектурное отличие — как лежат данные

	MonoBehaviour	ECS DOTS
Где данные	В managed-объекте на куче	В нативных непрерывных массивах
Где логика	В Update() того же класса	В отдельной System, без привязки к объекту
Как вызывается логика	Виртуальный вызов на каждом объекте	Один проход системы по массиву
GC	Активный, фриззы возможны	Не трогает игровые данные
Параллелизм	Однопоточный по умолчанию	Job System + Burst, все ядра CPU

Это **не косметическое отличие**. Это другая архитектура памяти, которая позволяет CPU работать в режиме «массовой обработки» — за то же время сделать в разы больше работы.

Почему именно так получается — ниже.



Memory Layout: AoS vs SoA

Это **ключ ко всему**. Понимание этой разницы — главное, что нужно вынести из статьи.

Array of Structures (AoS) — то, что делает MonoBehaviour

В памяти каждый объект **Enemy** лежит **целиком как блок**:

```
[Enemy 0: health, speed, transform, ai, audio, ... 200 байт]
[Enemy 1: health, speed, transform, ai, audio, ... 200 байт]
[Enemy 2: health, speed, transform, ai, audio, ... 200 байт]
...
```

Когда система обработки урона хочет прочитать **только Health** у всех врагов, процессор тащит из памяти **200 байт на каждого**, чтобы выковырять 4 байта (**float**). Остальные 196 байт — мусор, который засоряет кэш.

Современный CPU читает память **cache line'ами по 64 байта**. На один cache line в AoS помещается **меньше одного целого объекта**. Чтобы пройти 500 врагов — 500+ cache misses подряд. Каждый cache miss = ~100 циклов простоя CPU.

Structure of Arrays (SoA) — то, что делает ECS

В ECS компоненты одного типа лежат в памяти **поряд, отдельным массивом**:

```
Health:      [100][50][120][80][200][...500 значений...]  ← один непрерывный
массив float
MoveSpeed:   [10][12][8][11][9][...500 значений...]        ← другой массив
LocalTransform: [...500 transform-ов подряд...]
```

Теперь когда система урона хочет прочитать **Health** всех врагов, она проходит **один плотный массив float'ов**. На 64-байтный cache line помещается **16 значений Health**. Один cache miss → 16 объектов обработано. CPU работает на полной скорости предсказания.

Результат: **на больших объёмах данных ECS в 5–20× быстрее** только за счёт этого. Никакой магии — просто памятью пользуются правильно.

Virtual Dispatch и GC — два других убийцы перформанса

Кроме памяти, MonoBehaviour несёт ещё две «налоговые» нагрузки.

Virtual dispatch на каждом Update()

Update() у **MonoBehaviour** — виртуальный метод. Unity проходит по всему списку активных скриптов и **через vtable вызывает** каждый Update. Каждый вызов — branch, который CPU может не предугадать. На тысячах объектов это десятки тысяч непредсказуемых переходов.

В ECS — **одна система = один проход по массиву данных**. Никаких vtable. Компилятор знает форму данных и может агрессивно оптимизировать.

GC pressure от managed-классов

Каждый **MonoBehaviour** — managed-объект. Создание, удаление, любая **new List<>()** — мусор для GC. **GC stop-the-world** длится **миллисекунды** и виден игроку как «фриз».

В ECS компоненты — **struct**-ы. Сидят в нативных непрерывных буферах под управлением **EntityManager**. **GC их не трогает в принципе**. Никаких аллокаций в горячем пути, никаких фризов.

Совокупно: убирая AoS + virtual + GC, ECS получает **5–20× ускорение** на массовых сценариях. Не на любых — на **массовых**. Об этом ниже.

Когда брать ECS, а когда — нет

ECS — мощный, но **не бесплатный**. У него есть три налога:

- 1. **Кривая обучения**: ~2 недели полноценного onboarding'a для опытного Unity-разработчика
- 2. **Многословность**: больше boilerplate'a на простые задачи (Authoring + Baker + Component + System вместо одного MonoBehaviour)
- 3. **Совместимость**: Unity Physics → DOTS Physics, Animator → custom анимация, многие asset-store пакеты вообще не работают в ECS-мире

Решение «брать или нет» — это про **компромисс между этими налогами и выигрышем в производительности**.

Берите ECS	Не берите ECS
100+ однотипных объектов (враги, projectiles, particles)	UI и меню — оставляйте Mono + UI Toolkit
Жёсткие perf-таргеты: mobile, VR, Switch	Сложная логика на единственном объекте (GameManager)
RTS, survivors, factory games, swarm AI, симуляции	Прототип без perf-требований — overhead не окупится
В ТЗ слова «60 FPS на iPhone 8» / «1000 unit'ов одновременно»	Команда не готова инвестировать 2 недели в onboarding
Игра живёт долго и масштабируется (LiveOps, контент-патчи)	Тяжёлая зависимость от asset-store пакетов

Правило большого пальца: если можно показать «у нас будет 100+ X одновременно» — это серьёзный аргумент за ECS. Иначе сначала измеряйте профайлером, потом решайте.

Hybrid подход — реальность production

В моём pet-проекте структура такая:

- **90% кода — обычный MonoBehaviour**: UI, меню, метагейм (коллекция карт, BattlePass, фишинг)
- **10% — ECS DOTS**: только Survival-фича, где боль перформанса (массовые враги, projectiles, damage scan)

Между мирами — **бриджи**:

- Joystick (Mono, Canvas) пишет **MoveDirection** в ECS-singleton через статический класс
- HP-bar (Mono, World Space Canvas) читает **Health** и **MaxHealth** из ECS-entity игрока каждый кадр
- ECS damage-system эмитит **DamageEvent** entities, Mono-виджет в **LateUpdate** потребляет их и показывает floating «-5»

Это **самый частый production-сценарий**. Никто не переписывает весь проект на ECS — добавляют **точечно туда, где боль**. Моно-стек прекрасно справляется с UI и логикой меню, ECS закрывает массовку.

Что дальше в серии

1. **ECS DOTS: когда стандартный Unity сдаётся** ← вы здесь
2. **ECS внутри: Components, Systems, Baker** — подробный разбор всех типов компонентов, паттернов Authoring → Baker, ISystem vs SystemBase, EntityCommandBuffer, типичные ошибки
3. **Jobs: параллелизм без race conditions** — IJob / IJobEntity, Schedule vs ScheduleParallel, ComponentLookup, race conditions и как их обходить
4. **Burst: магия нативного кода в C#** — что Burst компилирует, что ему нельзя, benchmark до/после на реальном проекте

Если делал хайлоад в Unity — поделись опытом в комментариях. Где ECS оправдан, а где это overkill?

References

- **Unity DOCs — Entities package** <https://docs.unity3d.com/Packages/com.unity.entities@latest>
- **Unity DOTS Sample** <https://github.com/Unity-Technologies/EntityComponentSystemSamples>
- **Darkounity — What is Unity DOTS in 2026** <https://darkounity.com/blog/what-is-unity-dots-in-2026>

Формат поста для LinkedIn

Структура поста (по образцу серии Aleksandr Emtsev):

1. **Заголовок** — короткий и провокационный: «ECS DOTS: когда стандартный Unity сдаётся»
2. **Вступительный текст в ленте** — части «Зачем вообще ECS DOTS» + «Контекст» (см. выше). Это то, что читатель видит до клика «развернуть».
3. **Вложение (article preview card)** — полный текст статьи: разница архитектур, AoS vs SoA, virtual dispatch / GC, когда брать, hybrid подход.
4. **Картинка (cover image)** — сгенерированный спрайт-концепт ECS (см. описание ниже).

Описание картинки (для генерации)

Cover image для поста — концептуальная иллюстрация **AoS vs SoA**. Идея для генерации (Midjourney / ChatGPT image):

Two-column comparison diagram, programming/architecture style, clean technical illustration. Left column titled "MonoBehaviour (AoS)": vertical stack of identical boxes labeled "Enemy 0", "Enemy 1",

"Enemy 2", each box contains coloured cells "health", "speed", "transform", "ai", "audio". Highlight one cell ("health") in each box with red outline — show that to read just "health" you must jump through full structs. Right column titled "ECS DOTS (SoA)": three separate long horizontal stripes labeled "Health array", "Speed array", "Transform array". Each stripe filled with same-coloured cells in a row. Highlight the entire "Health array" stripe with green outline — show contiguous memory access. At the bottom: small caption "Cache line: 64 bytes" with a horizontal bracket spanning ~16 cells on the SoA Health stripe and ~0.3 of one Enemy box on the AoS side. Style: flat design, two accent colors (red/green), technical but accessible, optimised for LinkedIn thumbnail viewing on mobile.

Альтернатива — взять готовую диаграмму с darkounity.com (с атрибуцией) или нарисовать в Figma / Excalidraw за 10 минут.